

DATA SCIENCE NOTES

1. OVERFITTING

Overfitting occurs when a machine learning model learns the training data too well, including its noise and random fluctuations. The model performs very well on training data but poorly on new unseen data.

Signs of overfitting: High training accuracy, low test accuracy.

Solutions: Cross-validation, regularization, more training data, dropout (for neural networks), pruning (for decision trees).

2. UNDERFITTING

Underfitting occurs when a model is too simple to capture the underlying patterns in the data.

Signs: Low training accuracy and low test accuracy.

Solutions: Use more complex model, add more features, reduce regularization, train longer.

3. BIAS-VARIANCE TRADEOFF

Bias: Error from wrong assumptions in the model.

Variance: Error from sensitivity to small changes in training data.

High Bias = Underfitting. High Variance = Overfitting.

Goal: Find the sweet spot with low bias AND low variance.

4. LOGISTIC REGRESSION

Used for binary classification problems (Yes/No, 0/1).

Uses sigmoid function to output probabilities between 0 and 1.

Formula: $P(y=1) = 1 / (1 + e^{(-z)})$

where $z = b_0 + b_1 * x_1 + b_2 * x_2 + \dots$

Applications: Spam detection, disease prediction, credit scoring.

5. LINEAR REGRESSION

Used to predict continuous numerical values.

Finds the best fit line through data points.

Formula: $y = mx + b$

where m = slope, b = intercept.

Evaluation metrics: MSE, RMSE, MAE, R-squared.

6. DECISION TREES

A tree-like model that makes decisions based on feature values.

Each internal node = a feature test.

Each branch = outcome of the test.

Each leaf node = final prediction.

Advantages: Easy to interpret, handles non-linear data.

Disadvantages: Prone to overfitting.

7. RANDOM FOREST

An ensemble of many decision trees.

Each tree is trained on a random subset of data and features.

Final prediction = majority vote (classification) or average (regression).

More accurate and less prone to overfitting than a single tree.

8. PRECISION AND RECALL

Precision = $TP / (TP + FP)$

How many predicted positives are actually positive?

Use when False Positives are costly (e.g. spam filter).

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

How many actual positives were correctly predicted?

Use when False Negatives are costly (e.g. cancer detection).

$$\text{F1 Score} = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$$

Harmonic mean of precision and recall.

9. CONFUSION MATRIX

A table showing actual vs predicted classifications.

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

TP = True Positive (correctly predicted positive)

TN = True Negative (correctly predicted negative)

FP = False Positive (predicted positive but actually negative)

FN = False Negative (predicted negative but actually positive)

10. PRINCIPAL COMPONENT ANALYSIS (PCA)

A dimensionality reduction technique.

Transforms data into a new coordinate system.

New axes (principal components) capture maximum variance.

Used for: Visualization, noise reduction, feature extraction.

Steps:

1. Standardize the data

2. Compute covariance matrix
3. Find eigenvectors and eigenvalues
4. Select top k eigenvectors
5. Transform data

11. K-MEANS CLUSTERING

Unsupervised learning algorithm for grouping data.

Steps:

1. Choose k number of clusters
2. Randomly place k centroids
3. Assign each point to nearest centroid
4. Recalculate centroids
5. Repeat until centroids stop moving.

Use elbow method to find best k.

12. GRADIENT DESCENT

Optimization algorithm used to minimize the loss function.

Moves in the direction of steepest descent.

Learning rate controls step size.

Too high learning rate = overshooting.

Too low learning rate = very slow convergence.

Types: Batch, Stochastic (SGD), Mini-batch.

13. NEURAL NETWORKS

Inspired by the human brain.

Layers: Input layer, Hidden layers, Output layer.

Each node = neuron with activation function.

Common activation functions: ReLU, Sigmoid, Tanh, Softmax.

Training: Forward pass → Calculate loss → Backpropagation → Update weights.

14. CROSS VALIDATION

Technique to evaluate model performance reliably.

K-Fold: Split data into k parts, train on k-1, test on 1, repeat k times.

Prevents overfitting to a single train/test split.

Common values: k=5 or k=10.

15. FEATURE ENGINEERING

Process of creating new features from raw data.

Techniques: One-hot encoding, normalization, standardization, binning, log transformation, polynomial features.

Good features improve model performance significantly.

16. REGULARIZATION

Technique to prevent overfitting by penalizing complexity.

L1 (Lasso): Adds absolute value of weights to loss. Can zero out features.

L2 (Ridge): Adds squared weights to loss. Shrinks weights toward zero.

ElasticNet: Combination of L1 and L2.

17. SUPPORT VECTOR MACHINE (SVM)

Finds the hyperplane that best separates classes.

Maximizes the margin between classes.

Support vectors = data points closest to the hyperplane.

Kernel trick: Maps data to higher dimensions for non-linear separation.

Common kernels: Linear, RBF (Radial Basis Function), Polynomial.

18. NAIVE BAYES

Probabilistic classifier based on Bayes theorem.

Assumes features are independent (naive assumption).

Formula: $P(\text{class}|\text{features}) = P(\text{features}|\text{class}) * P(\text{class}) / P(\text{features})$

Fast, works well with text classification.

Types: Gaussian, Multinomial, Bernoulli.

19. TRAIN TEST SPLIT

Dividing dataset into training and testing sets.

Common split: 80% train, 20% test.

Purpose: Evaluate model on unseen data.

Always set `random_state` for reproducibility.

20. NORMALIZATION VS STANDARDIZATION

Normalization: Scale values to range [0,1].

Formula: $(x - \min) / (\max - \min)$

Use when: Algorithm needs bounded inputs (e.g. neural networks).

Standardization: Scale to mean=0, std=1.

Formula: $(x - \text{mean}) / \text{std}$

Use when: Data follows normal distribution (e.g. SVM, PCA).